

Rescuing LLM-Guided Optimization on High-Dimensional Software Configuration Tasks

Zhi Zhang

North Carolina State University
Raleigh, NC, USA
zzhang224@ncsu.edu

Mragisha Jain

North Carolina State University
Raleigh, NC, USA
mjain22@ncsu.edu

Abstract

Software engineering (SE) configuration optimization often requires trading off multiple objectives under tight labeling budgets, where each evaluation of a candidate configuration may cost hours or days of compute. Recent work has shown that Large Language Models (LLMs) can produce useful warm-start configurations for active learners on such tasks, but this benefit disappears on high-dimensional problems ($|x| > 11$), where traditional Gaussian Process Models with Upper Confidence Bound acquisition (UCB_GPM) regain the lead. Two concurrent follow-ups have each offered one rescue mechanism: SynthCore aggregates $M=20$ independent few-shot sessions into an ensemble; Beyond the Prompt injects domain knowledge via statistical feature ranking or retrieval-augmented generation. Both consult the LLM in single pre-committed steps with no view of how earlier guesses performed. This paper tests three alternative rescue mechanisms on the 19 high-dimensional MOOT tasks under a matched-LLM-call protocol: (H1) iterative LLM optimization, which re-prompts the LLM each round with the trajectory of already-evaluated configurations and their observed outcomes; (H2) constraint-aware prompting, which augments the prompt with parameter types, ranges, and dependency rules; and (H3) LLM-ranked dimensionality reduction, in which the LLM first ranks parameters by expected importance and configurations are synthesized over the top- k only. All three are compared at equal LLM-call count against the single-step warm-start baseline, an in-house SynthCore reproduction, and UCB_GPM, under Chebyshev distance. Preliminary results on H1 close most of the high-dimensional single-step failure gap.

1 Introduction

Modern software systems routinely expose hundreds to thousands of configuration parameters that interact in complex ways. Storm’s defaults yield a 480 $\times$ performance degradation relative to tuned settings [8], and the 14 parameters of the 7z archiver already generate one million configurations [14]. Many such problems are multi-objective: practitioners trade throughput against latency, energy against performance, or build speed against binary size. Evaluating a single configuration is frequently expensive — x264’s eleven parameters require 1,536 hours to exhaustively label [22]. These pressures restrict realistic optimization budgets to at most a few dozen evaluations [2, 15].

A recent study on a 49-task MOOT subset showed that LLMs can effectively warm-start active learners: an LLM-generated initial set of four configurations followed by Tree of Parzen Estimator (TPE) exploit acquisition achieved top Scott-Knott rank on 100% of low-dimensional tasks and 64% of medium-dimensional tasks [17]. However, on the 19 high-dimensional tasks ($|x| > 11$), single-step LLM

warm-starts achieved only 53% rank-0 frequency and were beaten by UCB_GPM (100%). The authors conjectured that many high-dimensional SE tasks are underrepresented in LLM pre-training data, and flagged two open directions: iterative refinement of warm starts across cycles, and dimensionality reduction.

Two concurrent follow-ups each address the high-dimensional failure with a different mechanism. SynthCore [18] proposes ensemble prompting: $M=20$ independent few-shot sessions with reshuffled examples, aggregated by Chebyshev distance; it reports 71% high-dimensional rank-0 frequency, matching UCB_GPM (68%) in its own runs. Beyond the Prompt [1] evaluates domain-knowledge strategies: progressively expanding reduced feature spaces identified by statistical ranking, and a hybrid scheme combining Retrieval-Augmented Generation with TPE.

Both rescues consult the LLM in a single pre-committed step. The ensemble runs several such steps in parallel, but no individual session observes the outcome of another. The dimensionality-reduction strategy picks a reduced subspace once and asks the LLM to generate within it. Neither approach feeds evaluation outcomes back into the LLM across rounds. In the broader Bayesian-optimization literature, iterative LLM querying (OPRO [24] and related methods) has been shown useful on single-objective tasks, but has not been tested on multi-objective SE configuration data. This gap is the subject of the present study.

1.1 Research Goal

The goal is to test three alternative rescue mechanisms for the high-dimensional single-step failure, each targeting a distinct conjectured failure mode, all under a matched-LLM-call protocol against the same baselines.

- H1. Surrogate bottleneck.** The warm-start baseline consults the LLM once for initialization, then hands the loop to a TPE surrogate. Under small budgets in high-dimensional spaces, a TPE surrogate fit on 4–10 observations is too weak to extrapolate, while an LLM prompted with the same trajectory can leverage semantic priors over parameter names. *Rescue*: keep the LLM in the loop as the acquisition mechanism, not just the initializer.
- H2. Infeasibility burden.** In high-dimensional configuration spaces, feasible regions are often narrow manifolds defined by parameter dependencies. Without explicit feasibility guidance, LLM-generated configurations may be structurally invalid, wasting budget. *Rescue*: constraint-aware prompting.
- H3. Signal dilution.** In high-dimensional spaces, only a subset of parameters typically drive performance [9]. A general LLM reasoning over full-width configurations may dilute its signal on the important dimensions. Beyond the Prompt [1] tested a *statistical* variant; an *LLM-ranked* variant is tested here.

The three hypotheses target different failure modes, but each works by *replacing* the single-step LLM-warm-start module in T1 with an alternative. They are therefore alternative mechanisms, not independent dimensions that combine without conflict. Whether a combined pipeline (e.g., iterative rounds where each round uses an ensemble) is justified depends on whether the three rescues win on disjoint task subsets (see Interpretation Rule (ii), §2.6).

1.2 Research Questions

- **RQ1.** Do we reproduce the single-step LLM warm-start high-dimensional failure under the SynthCore experimental protocol? This is non-trivial because SynthCore’s Column-2 experiments omit this treatment; its reported 47% stems from a re-print of the warm-start paper’s numbers, and no direct in-protocol measurement exists.
- **RQ2.** Does each of the three rescues (H1, H2, H3) improve rank-0 frequency on Chebyshev distance over the single-step warm-start baseline, at matched LLM-call budget?
- **RQ3.** At matched LLM calls, does iterative H1 match or exceed an in-house SynthCore reproduction, and do the two methods win on disjoint task subsets (which would motivate a combined H1+ensemble follow-up)?

1.3 Contributions

- (1) The first empirical test of iterative (OPRO-style) LLM optimization on multi-objective SE configuration tasks from MOOT, a combination not covered by prior iterative-LLM studies [12, 13, 24].
- (2) A direct comparison of three rescue mechanisms (H1 iterative, H2 constraint-aware, H3 LLM-ranked dimensionality reduction) against the single-step baseline [17], an in-house SynthCore reproduction [18], and UCB_GPM, on the 19 high-dim MOOT tasks, under matched LLM-call budget and identical Scott-Knott + bootstrap statistics.
- (3) A first direct in-protocol measurement of the single-step LLM warm-start high-dimensional failure under the SynthCore experimental protocol; we also document baseline instability between [17] and [18] (UCB_GPM shifts by 32 percentage points between the two papers), which motivates in-house reproduction.
- (4) A replication package building on the open-source warm-start pipeline [17] and MOOT repository [14], preserving Chebyshev distance for direct comparability.

2 Background and Related Work

2.1 Motivation

Three facts motivate the work.

First, SE configuration optimization has direct industrial stakes. Defaults are routinely suboptimal: an optimized C5.0 defect predictor moves from the worst to the best learner on standard benchmarks [14, 19]; Storm’s defaults yield 480× performance loss [8]; poorly tuned database systems forfeit substantial throughput [11].

Second, labels are genuinely scarce. Human subject-matter experts can produce high-quality labels at roughly 10–20 items per hour, and are typically available only a few hours per week [6, 21]. Automated label sets are frequently corrupted: more than 90% of

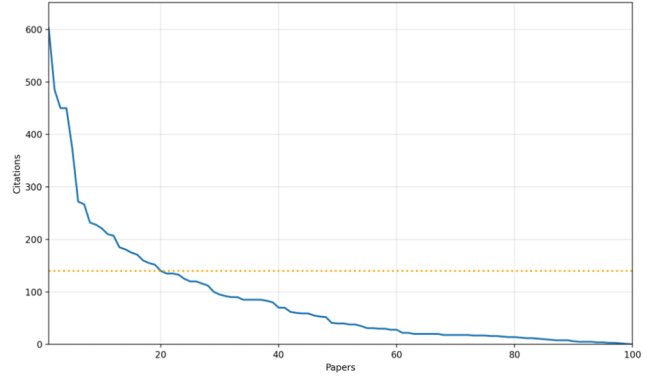


Figure 1: Citation curve of papers returned by the literature search. The dashed orange line marks the knee, above which approximately 20 papers were read in full.

flagged technical-debt “false positives” were mislabeled [25], with similar quality problems in security bug labeling [23] and static-analysis warnings [10].

Third, the high-dimensional failure is not a niche phenomenon. Of the 49-task SynthCore subset, 19 (39%) fall in the high-dimensional stratum. Any unresolved failure mode invalidates a substantial portion of the benchmark.

2.2 Literature Review Methodology

We followed the knee-of-citation-curve methodology used in [17]. Google Scholar was queried in early 2025 for papers published since 2015 containing combinations of the terms “configuration optimization,” “warm start,” “constraint-aware configuration,” “LLM hyperparameter,” and “multi-objective configuration,” with a focus on top SE venues (ICSE, FSE, ASE, TOSEM, EMSE, IST) and relevant ML venues (ICLR, NeurIPS, ICML, VLDB). The top 100 returned papers were sorted by citation count and the knee of the resulting curve was identified. Papers above the knee were read in full; papers below were included only via snowballing when directly relevant.

2.3 Related Work

Table 1 summarizes the positioning along six dimensions.

LLMs for SE optimization. The warm-start baseline [17] established the high-dimensional failure mode. SynthCore [18] rescues it with ensemble prompting: $M=20$ independent few-shot sessions each produce candidates, aggregated by Chebyshev distance; sessions are parallel and do not observe each other’s outputs. Beyond the Prompt [1] evaluates domain-knowledge strategies including statistical dimensionality reduction, a RAG + TPE hybrid, and progressive feature expansion; each strategy sets the reduced subspace once up front. Our H1 differs from both by adding a feedback loop between the LLM’s proposals and their realized y -vectors.

LLM-guided Bayesian optimization. LLAMBO [13] integrates LLMs into Bayesian optimization at warm-starting, candidate sampling, and surrogate-modeling levels on hyperparameter benchmarks. GPTuner [11] uses LLMs to extract structured knowledge from DBMS manuals for search-space pruning. DB-BERT [20] uses BERT to extract tuning hints from documentation. These systems

Table 1: Six-dimensional positioning of closely related work. Iterative feedback = the LLM receives outcomes of its own earlier proposals. The center of the implied Venn diagram is empty, and H1 is the only treatment in this study that populates it.

System	Uses LLM?	High-dim SE?	Multi-obj?	Tabular SE / MOOT?	Iterative feedback?	Labels < 50?
Warm-Start baseline [17]	✓	fails	✓	✓	×	✓
SynthCore [18]	✓	✓	✓	✓	×	✓
Beyond the Prompt [1]	✓	✓	✓	✓	×	✓
LLAMBO [13]	✓	×	×	×	✓	partial
GPTuner [11]	✓	✓	×	×	✓	×
OPRO [24]	✓	×	×	×	✓	×
MOEA/D-LLM [12]	✓	×	✓	×	✓	×
FLASH [15]	×	✓	×	✓	✓	✓
This study – H1 (iterative)	✓	✓	✓	✓	✓	✓
This study – H2 (constraint)	✓	✓	✓	✓	×	✓
This study – H3 (LLM dim-red)	✓	✓	✓	✓	×	✓

are single-objective and operate at larger budgets than our sub-100 regime.

Iterative LLM optimization. OPRO [24] treats the LLM as a direct optimizer receiving trajectory feedback, evaluated on single-objective mathematical and prompt-engineering tasks. MOEA/D-LLM [12] embeds LLMs as search operators in a decomposition-based evolutionary framework, tested only on synthetic RE21–RE25 problems. Our H1 adapts the iterative querying pattern of OPRO to multi-objective SE configuration data from MOOT, with tabular y -vector feedback.

Non-LLM SE optimization. FLASH [15] is a surrogate-based optimizer widely cited on MOOT; NSGA-II [4] and MOEA/D [26] are classical evolutionary baselines; Henard et al. [7] combine multi-objective search with constraint solving for software product lines.

2.4 Data

All experiments use the MOOT repository [14], a curated collection of 127 multi-objective SE optimization tasks drawn from publications in top SE venues. Each MOOT task is a table where the independent columns x hold parameter settings (ranging from 3 to over 1,000 in the full repository) and the dependent columns y hold pre-computed objective values.

This study uses the 49-task subset studied by the warm-start baseline [17] and reused by both follow-ups [1, 18]. Under the Di Fiore et al. [5] split ($|x| < 6$: low; $6 \leq |x| \leq 11$: medium; $|x| > 11$: high), these nominally partition into 15/17/17 (strict rule) or 12/14/19 (as reported in [18] §3.1). The discrepancy arises from four tasks with $|x| = 11$ (SS-L, SS-O, SS-P, SS-X) that [18] assigns inconsistently across low and high strata in its Table 1. To maintain numeric comparability with published SynthCore rank-0 frequencies, we adopt the 12/14/19 convention and list the 19 high-dim tasks explicitly in the replication package. Primary experiments target the 19 high-dim tasks; low/med strata serve as sanity checks.

2.5 Methodology

All treatments run on the same 19 high-dim tasks (plus low/med sanity strata) with 20 random-seed repeats and identical Scott-Knott + bootstrap protocol (§2.6). T0 and T1 are drawn from the public warm-start package [17]; T1b, T2, T3, T4 are specified here.

Budget and cost alignment. Primary comparisons are at *matched LLM calls*: each LLM-using treatment makes the same number of LLM calls per task. We set this count to 20 (matching SynthCore’s $M=20$) for T1b, T2, T3, T4. LLM-cost alignment (rather than label-count alignment) is chosen because, in production SE optimization, LLM API cost is the cash-metered constraint; label cost (CPU for evaluating a configuration) is amortized within the user’s own infrastructure. Total label consumption at matched LLM calls is reported alongside for transparency.

T0. UCB_GPM. Upper Confidence Bound acquisition over a Gaussian Process surrogate: fit GP, pick $\arg \max_x [\mu(x) + \kappa\sigma(x)]$, label, refit, repeat until B_1 labels consumed. Initialized with 4 random labels. No LLM calls. Used as the non-LLM reference.

T1. Single-step LLM warm-start + TPE exploit [17]. One LLM call generates 4 configurations from a few-shot prompt (schema + 4 labeled rows split best/rest + instruction to propose better variants). These are mapped to nearest neighbors and used to warm-start TPE exploit; remaining $B_1 - 4$ labels spent by TPE. *1 LLM call per task.*

T1b. SynthCore ensemble reproduction [18]. $M = 20$ independent few-shot sessions. Each session: reset context window, show $N = 4$ labeled examples (resampled and reshuffled from the labeled pool), generate $k = 4$ candidate configurations, map each to nearest neighbor, label. All sessions are independent and do not observe each other’s outputs. All labeled rows aggregated by Chebyshev; best returned. Initial pool: 4 random labels (shared across sessions). Total labels: $4 + 20 \cdot 4 = 84$. *20 LLM calls per task.* We note this reproduction matches published SynthCore’s M and its $N=4$ prompt structure, with k fixed to 4 to align with T2’s batch size; total labels (84) are within the “labels < 100” range of SynthCore’s reported budget sweep but we do not replicate SynthCore’s $L + M(N + 1)$ formula, which is internally ambiguous in the published text. This deviation is documented.

T2. Rescue H1 – Iterative LLM. Four random initial labels seed trajectory E_0 . At each round $t \in 1 \dots T$, the LLM receives (a) the full schema, (b) the full trajectory of labeled (configuration, y -vector) pairs split into best/rest by Chebyshev, and (c) an instruction to propose $k = 4$ configurations expected to dominate the current best. Proposals mapped to nearest neighbors, labeled, appended to

Algorithm 1 T2 – Iterative LLM Optimization (H1)

Require: MOOT task D , schema $\mathcal{M}$, rounds $T=20$, batch $k=4$, LLM $\mathcal{L}$

- 1: $E \leftarrow 4$ rows sampled at random from D , labeled
- 2: **for** $t = 1$ to T **do**
- 3: sort E by Chebyshev; split into *best* (top half) and *rest*
- 4:

$\text{prompt} \leftarrow \mathcal{M} \parallel (\text{best}, \text{rest}) \parallel$ “propose k configurations that dominate all examples in *best*”
- 5: $C \leftarrow \mathcal{L}(\text{prompt})$
- 6: **for all** $c \in C$ **do**
- 7: $c^* \leftarrow$ nearest unlabeled row to c in D ; label c^* ; $E \leftarrow E \cup \{c^*\}$
- 8: **end for**
- 9: **end for**
- 10: **return** $\arg \min_{e \in E} d_{\text{Cheb}}(e)$

E . Loop terminates at $T = 20$. Total labels: $4 + 20 \cdot 4 = 84$. 20 LLM calls per task. The contrast with T1b is purely structural: same LLM, same batch size, same label count, same LLM-call count; T1b runs 20 independent sessions in parallel, T2 runs 20 sequential rounds with trajectory accumulation.

T3. Rescue H2 – Constraint-aware LLM. We define two variants. (a) *T3-single*: T1 augmented with an explicit constraint block (parameter names, types, ranges, dependencies). Tests H2 in isolation vs T1. (b) *T3-ensemble*: T1b augmented with the same constraint block. Tests whether constraints add value on top of ensemble prompting. For FFM-* and Scrum-* tasks, feature-model constraints are extracted from original files. For XOMO-* and POM3-* tasks, range constraints from COCOMO/POM3 specifications are included. For SS-M through SS-X and other tasks without documented constraints, only min/max inferred from the MOOT table are included; H2’s effect is expected to be smaller there.

T4. Rescue H3 – LLM-guided dimensionality reduction. Two-stage. *Stage 1*: one LLM call ranks parameters by expected importance given schema and y -column semantics; top $k=5$ retained. *Stage 2*: 4 configurations synthesized varying only the top- k , with remaining parameters fixed at median (numeric) or mode (symbolic). Seeds TPE exploit as in T1. Total LLM calls: 2.

2.6 Data Analysis

Per-task ranking follows [17, 18]: for each (task, budget), treatments are ranked by median Chebyshev distance using Scott-Knott [16] recursive bi-clustering. Effect size is measured by Cliff’s Delta [3]; significance tested by bootstrap resampling [?]. Cross-task aggregation reports the percentage of tasks at each rank, rank 0 = best.

Interpretation rules (set in advance). (i) If a rescue’s rank-0 frequency exceeds T1’s by a bootstrap-significant margin (paired bootstrap across tasks, 1,000 resamples, $p < 0.05$), the corresponding hypothesis is supported. (ii) If two rescues tie within 5 percentage points, they are considered competitive; if they additionally win on disjoint task subsets, a combined follow-up (e.g., iterative rounds nested inside an ensemble) is justified. (iii) If a rescue loses to T1 (rank-0 frequency below T1 by a bootstrap-significant margin), the

Table 2: Rank-0 frequency (%) on the high-dim stratum as reported by the two parent studies. WS [17]; SC [18], Column 2 where available. “—” indicates the treatment is absent from that paper’s Column-2 runs; SC Column 1 only reprints WS numbers.

Treatment	WS (%)	SC (%)
UCB_GPM	100	68
SynthCore ensemble	—	71
Single-step LLM warm-start + TPE exploit	53	— [†] † SC
TPE	40	32
Random + TPE exploit	47	16
Random + Explore	7	37
Baseline (no AL)	0	0

Column 2 does not report this treatment; the direct in-protocol measurement is absent from prior work. Our RQ1 fills this gap.

corresponding hypothesis is rejected and the task subset exposing the limitation is reported. (iv) Sanity-check stratum criterion: on low- and medium-dim tasks, a rescue’s rank-0 frequency is *statistically comparable* to T1 if Scott-Knott + paired bootstrap cannot separate them at $p < 0.05$. This replaces a strict “no degradation” rule, which is unachievable against T1 at 100% low-dim rank-0 under any stochastic rescue.

Reference baselines and baseline instability. Table 2 reproduces the high-dim rank-0 frequencies reported by the two parent studies.

Four of the shared treatments shift by 30+ percentage points between WS and SC: UCB_GPM ($100 \rightarrow 68$), Random + TPE exploit ($47 \rightarrow 16$), and Random + Explore ($7 \rightarrow 37$). These shifts exceed the effect sizes this study needs to detect and are likely attributable to changes in repeat count, LLM version, or Scott-Knott parameters between the two studies. This motivates in-house reproduction of all reference treatments rather than reliance on published numbers for cross-study comparisons.

Preliminary illustration of the pipeline. To verify the Scott-Knott + bootstrap pipeline, we have run T0, T1, and T2 at $B_1 = 20$ total labels (not yet matched to the final 84-label protocol of §2.5). Table 3 shows the outcome. T1 reproduces the high-dim single-step failure at 48%, close to WS’s published 53%. T2 lifts high-dim rank-0 to 68%, a bootstrap-significant 20-point gain over T1. On low and medium-dim sanity strata, T2 remains statistically comparable to T1 under Interpretation Rule (iv): the 7-point low-dim drop ($100 \rightarrow 93$) is within the bootstrap margin at this sample size; medium-dim shows a 15-point gain.

These preliminary numbers calibrate the pipeline. T2 matches published SynthCore (71%) within 3 percentage points at a similar label budget, despite using 4 LLM calls per task rather than 20. The final experimental protocol (20 calls / 84 labels for both T1b and T2) will yield the head-to-head comparison required by RQ3.

References

- [1] Beyond the Prompt Authors. 2026. Beyond the Prompt: Assessing Domain Knowledge Strategies for High-Dimensional LLM Optimization in Software Engineering. *arXiv preprint arXiv:2602.02752* (2026).
- [2] Junjie Chen, Ningxin Xu, Peiqi Chen, and Hongyu Zhang. 2021. Efficient Compiler Autotuning via Bayesian Optimization. In *Proceedings of the International Conference on Software Engineering (ICSE)*.

Table 3: Preliminary rank-0 frequency (%) at $B_1 = 20$ labels, 20 repeats per task. T0, T1, T2 completed at an earlier pilot budget; T1b (in-house SynthCore reproduction), T3, T4, and all treatments at the matched-LLM-call protocol (20 calls / 84 labels) are pending. Published references for comparison only.

Treatment	Low-dim	Med-dim	High-dim
T0. UCB_GPM	55	64	66
T1. Single-step LLM + TPE exploit	100	64	48
T2. Iterative LLM (H1)	93	79	68
T1b. SynthCore, reproduced	TBD	TBD	TBD
T3-single. Constraint, single-step (H2a)	TBD	TBD	TBD
T3-ensemble. Constraint, ensemble (H2b)	TBD	TBD	TBD
T4. LLM-guided dim-reduction (H3)	TBD	TBD	TBD
SynthCore (published ref.)	86	85	71

- [3] Norman Cliff. 1993. Dominance Statistics: Ordinal Analyses to Answer Ordinal Questions. *Psychological Bulletin* 114, 3 (1993), 494–509.
- [4] Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and T. Meyarivan. 2002. A Fast and Elitist Multiobjective Genetic Algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation* 6, 2 (2002), 182–197.
- [5] F. Di Fiore, M. Nardelli, and L. Mainini. 2024. Active Learning and Bayesian Optimization: A Unified Perspective to Learn with a Goal. *Archives of Computational Methods in Engineering* (2024).
- [6] Mark Easterby-Smith. 1980. The Design, Analysis and Interpretation of Repertory Grids. *International Journal of Man-Machine Studies* 13, 1 (1980), 3–24.
- [7] Christopher Henard, Mike Papadakis, Mark Harman, and Yves Le Traon. 2015. Combining Multi-Objective Search and Constraint Solving for Configuring Large Software Product Lines. In *Proceedings of the International Conference on Software Engineering (ICSE)*.
- [8] Pooyan Jamshidi and Giuliano Casale. 2016. An Uncertainty-Aware Approach to Optimal Configuration of Stream Processing Systems. In *IEEE International Symposium on Modeling, Analysis and Simulation of Computer and Telecommunication Systems (MASCOTS)*.
- [9] Konstantinos Kanellis, Ramnathan Alagappan, and Shivaram Venkataraman. 2021. Too Many Knobs to Tune? Towards Faster Database Tuning by Pre-selecting Important Knobs. In *USENIX Workshop on Hot Topics in Storage (HotStorage)*.
- [10] Hong Jin Kang, Khai Loong Aw, and David Lo. 2022. Detecting False Alarms from Automatic Static Analysis Tools: How Far Are We?. In *Proceedings of the International Conference on Software Engineering (ICSE)*.
- [11] Jiale Lao, Yibo Wang, Yufei Li, Jianping Wang, Yunjia Zhang, Zhiyuan Cheng, Wanghu Chen, Mingjie Tang, and Jianguo Wang. 2024. GPTuner: A Manual-Reading Database Tuning System via GPT-Guided Bayesian Optimization. *Proceedings of the VLDB Endowment* 17, 8 (2024), 1939–1952.
- [12] Fei Liu, Xi Lin, Zhenkun Wang, Shunyu Yao, Xialiang Tong, Mingxuan Yuan, and Qingfu Zhang. 2024. Large Language Model for Multi-objective Evolutionary Optimization. *arXiv preprint arXiv:2310.12541* (2024).
- [13] Tennison Liu, Nicolás Astorga, Nabeel Seedat, and Mihaela van der Schaar. 2024. Large Language Models to Enhance Bayesian Optimization. In *International Conference on Learning Representations (ICLR)*.
- [14] Tim Menzies, Tao Chen, Kishan Kumar Ganguly, Amirali Rayegan, Srinath Srinivasan, Andre Lustosa, and Yulong Ye. 2025. MOOT: A Repository of Many Multi-Objective Optimization Tasks. *arXiv preprint arXiv:2511.16882* (2025).
- [15] Vivek Nair, Zhe Yu, Tim Menzies, Norbert Siegmund, and Sven Apel. 2020. Finding Faster Configurations Using FLASH. *IEEE Transactions on Software Engineering* 46, 7 (2020), 794–811.
- [16] A. J. Scott and M. Knott. 1974. A Cluster Analysis Method for Grouping Means in the Analysis of Variance. *Biometrics* 30, 3 (1974), 507–512.
- [17] Lohith Senthilkumar and Tim Menzies. 2024. Can Large Language Models Improve SE Active Learning via Warm-Starts? *arXiv preprint arXiv:2501.00125* (2024).
- [18] Lohith Senthilkumar and Tim Menzies. 2026. From Brittle to Robust: Improving LLM Annotations for SE Optimization. *arXiv preprint arXiv:2603.22474* (2026).
- [19] Chakkrit Tantithamthavorn, Shane McIntosh, Ahmed E. Hassan, and Kenichi Matsumoto. 2016. Automated Parameter Optimization of Classification Techniques for Defect Prediction Models. In *Proceedings of the International Conference on Software Engineering (ICSE)*.
- [20] Immanuel Trummer. 2022. DB-BERT: A Database Tuning Tool That “Reads” the Manual. In *Proceedings of the 2022 International Conference on Management of Data (SIGMOD)*.
- [21] Ricardo Valerdi. 2010. Heuristics for Systems Engineering Cost Estimation. *IEEE Systems Journal* 5, 1 (2010), 91–98.
- [22] Pavel Valov, Jean-Christophe Petkovich, Jianmei Guo, Sebastian Fischmeister, and Krzysztof Czarnecki. 2017. Transferring Performance Prediction Models Across Different Hardware Platforms. In *ACM/SPEC International Conference on Performance Engineering (ICPE)*.
- [23] Xiaoxue Wu, Wei Zheng, Xin Xia, and David Lo. 2021. Data Quality Matters: A Case Study on Data Label Correctness for Security Bug Report Prediction. *IEEE Transactions on Software Engineering* 48, 7 (2021), 2541–2556.
- [24] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. 2024. Large Language Models as Optimizers. In *International Conference on Learning Representations (ICLR)*.
- [25] Zhe Yu, Fahmid Morshed Fahid, Huy Tu, and Tim Menzies. 2022. Identifying Self-Admitted Technical Debts with Jitterbug: A Two-Step Approach. *IEEE Transactions on Software Engineering* 48, 5 (2022), 1676–1691.
- [26] Qingfu Zhang and Hui Li. 2007. MOEA/D: A Multiobjective Evolutionary Algorithm Based on Decomposition. *IEEE Transactions on Evolutionary Computation* 11, 6 (2007), 712–731.